8.2 Binary Trees

A ***binary tree*** is an ordered tree with the following properties:

1. Every node has at most two children.
2. Each child node is labeled as being either a ***left child*** or a ***right child***.
3. A left child precedes a right child in the order of children of a node.

The subtree rooted at a left or right child of an internal node v is called a ***left subtree*** or ***right subtree***, respectively, of v. A binary tree is ***proper*** if each node has either zero or two children. Some people also refer to such trees as being ***full*** binary trees. Thus, in a proper binary tree, every internal node has exactly two children. A binary tree that is not proper is ***improper***.

Example 8.6: *An important class of binary trees arises in contexts where we wish to represent a number of different outcomes that can result from answering a series of yes-or-no questions. Each internal node is associated with a question. Starting at the root, we go to the left or right child of the current node, depending on whether the answer to the question is "Yes" or "No." With each decision, we follow an edge from a parent to a child, eventually tracing a path in the tree from the root to a leaf. Such binary trees are known as* **decision trees**, *because a leaf position p in such a tree represents a decision of what to do if the questions associated with p's ancestors are answered in a way that leads to p. A decision tree is a proper binary tree. Figure 8.7 illustrates a decision tree that provides recommendations to a prospective investor.*

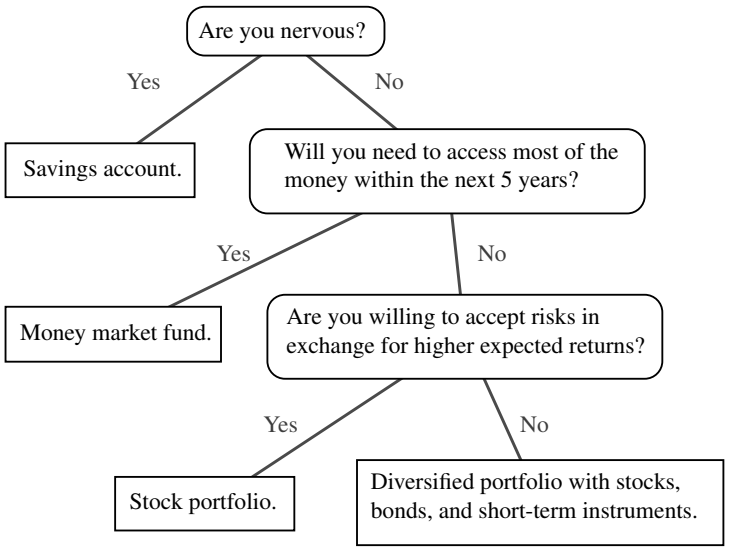


Figure 8.7: A decision tree providing investment advice.

Example 8.7: *An arithmetic expression can be represented by a binary tree whose leaves are associated with variables or constants, and whose internal nodes are associated with one of the operators $+$, $-$, $\times$, and $/$. (See Figure 8.8.) Each node in such a tree has a value associated with it.*

- *If a node is leaf, then its value is that of its variable or constant.*
- *If a node is internal, then its value is defined by applying its operation to the values of its children.*

An arithmetic expression tree is a proper binary tree, since each operator $+$, $-$, $\times$, and $/$ takes exactly two operands. Of course, if we were to allow unary operators, like negation ($-$), as in "$-x$," then we could have an improper binary tree.

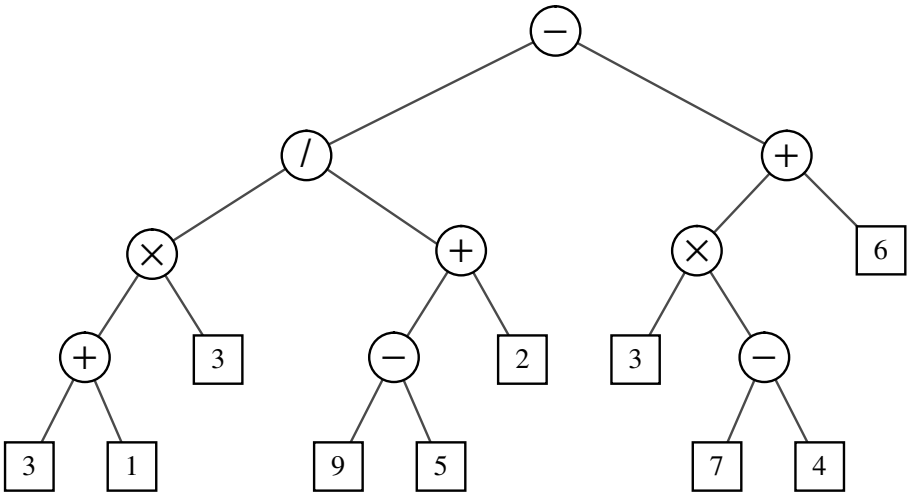


Figure 8.8: A binary tree representing an arithmetic expression. This tree represents the expression $((((3+1)\times 3)/((9-5)+2))-((3\times(7-4))+6))$. The value associated with the internal node labeled "/" is 2.

A Recursive Binary Tree Definition

Incidentally, we can also define a binary tree in a recursive way such that a binary tree is either empty or consists of:

- A node r, called the root of T, that stores an element
- A binary tree (possibly empty), called the left subtree of T
- A binary tree (possibly empty), called the right subtree of T

8.2.1 The Binary Tree Abstract Data Type

As an abstract data type, a binary tree is a specialization of a tree that supports three additional accessor methods:

T.left(p): Return the position that represents the left child of p, or None if p has no left child.

T.right(p): Return the position that represents the right child of p, or None if p has no right child.

T.sibling(p): Return the position that represents the sibling of p, or None if p has no sibling.

Just as in Section 8.1.2 for the tree ADT, we do not define specialized update methods for binary trees here. Instead, we will consider some possible update methods when we describe specific implementations and applications of binary trees.

The BinaryTree Abstract Base Class in Python

Just as Tree was defined as an abstract base class in Section 8.1.2, we define a new BinaryTree class associated with the binary tree ADT. We rely on inheritance to define the BinaryTree class based upon the existing Tree class. However, our BinaryTree class remains *abstract*, as we still do not provide complete specifications for how such a structure will be represented internally, nor implementations for some necessary behaviors.

Our Python implementation of the BinaryTree class is given in Code Fragment 8.7. By using inheritance, a binary tree supports all the functionality that was defined for general trees (e.g., parent, is_leaf, root). Our new class also inherits the nested Position class that was originally defined within the Tree class definition. In addition, the new class provides declarations for new abstract methods left and right that should be supported by concrete subclasses of BinaryTree.

Our new class also provides two concrete implementations of methods. The new sibling method is derived from the combination of left, right, and parent. Typically, we identify the sibling of a position p as the "other" child of p's parent. However, if p is the root, it has no parent, and thus no sibling. Also, p may be the only child of its parent, and thus does not have a sibling.

Finally, Code Fragment 8.7 provides a concrete implementation of the children method; this method is abstract in the Tree class. Although we have still not specified how the children of a node will be stored, we derive a generator for the ordered children based upon the implied behavior of abstract methods left and right.

```
class BinaryTree(Tree):
  """Abstract base class representing a binary tree structure."""

  # --------------------- additional abstract methods ---------------------
  def left(self, p):
    """Return a Position representing p's left child.

    Return None if p does not have a left child.
    """
    raise NotImplementedError('must be implemented by subclass')

  def right(self, p):
    """Return a Position representing p's right child.

    Return None if p does not have a right child.
    """
    raise NotImplementedError('must be implemented by subclass')

  # ---------- concrete methods implemented in this class ----------
  def sibling(self, p):
    """Return a Position representing p's sibling (or None if no sibling)."""
    parent = self.parent(p)
    if parent is None:                    # p must be the root
      return None                         # root has no sibling
    else:
      if p == self.left(parent):
        return self.right(parent)         # possibly None
      else:
        return self.left(parent)          # possibly None

  def children(self, p):
    """Generate an iteration of Positions representing p's children."""
    if self.left(p) is not None:
      yield self.left(p)
    if self.right(p) is not None:
      yield self.right(p)
```

Code Fragment 8.7: A BinaryTree abstract base class that extends the existing Tree abstract base class from Code Fragments 8.1 and 8.2.

8.2.2 Properties of Binary Trees

Binary trees have several interesting properties dealing with relationships between their heights and number of nodes. We denote the set of all nodes of a tree T at the same depth d as ***level*** d of T. In a binary tree, level 0 has at most one node (the root), level 1 has at most two nodes (the children of the root), level 2 has at most four nodes, and so on. (See Figure 8.9.) In general, level d has at most 2^d nodes.

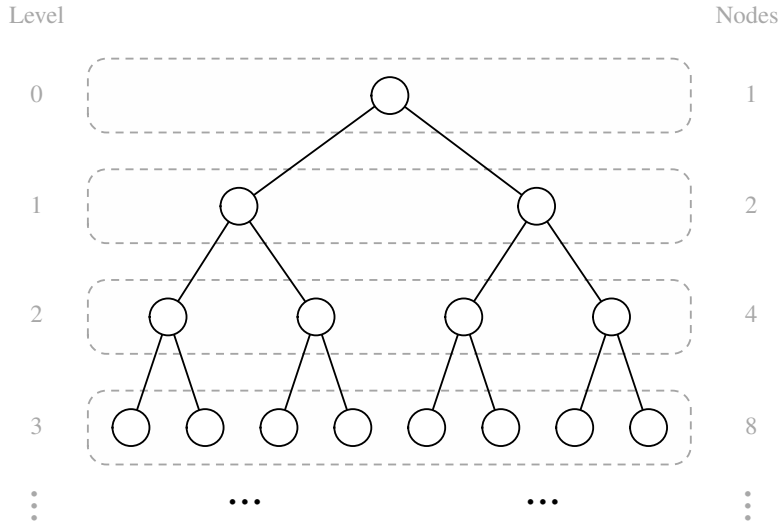


Figure 8.9: Maximum number of nodes in the levels of a binary tree.

We can see that the maximum number of nodes on the levels of a binary tree grows exponentially as we go down the tree. From this simple observation, we can derive the following properties relating the height of a binary tree T with its number of nodes. A detailed justification of these properties is left as Exercise R-8.8.

Proposition 8.8: *Let T be a nonempty binary tree, and let n, n_E, n_I and h denote the number of nodes, number of external nodes, number of internal nodes, and height of T, respectively. Then T has the following properties:*

1. $h+1 \leq n \leq 2^{h+1}-1$
2. $1 \leq n_E \leq 2^h$
3. $h \leq n_I \leq 2^h-1$
4. $\log(n+1)-1 \leq h \leq n-1$

Also, if T is proper, then T has the following properties:

1. $2h+1 \leq n \leq 2^{h+1}-1$
2. $h+1 \leq n_E \leq 2^h$
3. $h \leq n_I \leq 2^h-1$
4. $\log(n+1)-1 \leq h \leq (n-1)/2$

Relating Internal Nodes to External Nodes in a Proper Binary Tree

In addition to the earlier binary tree properties, the following relationship exists between the number of internal nodes and external nodes in a proper binary tree.

Proposition 8.9: *In a nonempty proper binary tree T, with n_E external nodes and n_I internal nodes, we have $n_E = n_I + 1$.*

Justification: We justify this proposition by removing nodes from T and dividing them up into two "piles," an internal-node pile and an external-node pile, until T becomes empty. The piles are initially empty. By the end, we will show that the external-node pile has one more node than the internal-node pile. We consider two cases:

Case 1: If T has only one node v, we remove v and place it on the external-node pile. Thus, the external-node pile has one node and the internal-node pile is empty.

Case 2: Otherwise (T has more than one node), we remove from T an (arbitrary) external node w and its parent v, which is an internal node. We place w on the external-node pile and v on the internal-node pile. If v has a parent u, then we reconnect u with the former sibling z of w, as shown in Figure 8.10. This operation, removes one internal node and one external node, and leaves the tree being a proper binary tree.

Repeating this operation, we eventually are left with a final tree consisting of a single node. Note that the same number of external and internal nodes have been removed and placed on their respective piles by the sequence of operations leading to this final tree. Now, we remove the node of the final tree and we place it on the external-node pile. Thus, the the external-node pile has one more node than the internal-node pile.

■

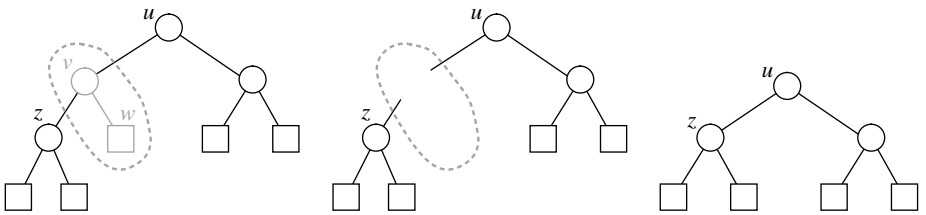


Figure 8.10: Operation that removes an external node and its parent node, used in the justification of Proposition 8.9.

Note that the above relationship does not hold, in general, for improper binary trees and nonbinary trees, although there are other interesting relationships that do hold. (See Exercises C-8.32 through C-8.34.)